

به نام خدا



دانشگاه تهران



دانشکده مهندسی برق و کامپیوتر

درس یادگیری تقویتی

تمرین اول

نام و نام خانوادگی	امیر غرقابی
شماره دانشجویی	۸۱۰۱۰۲۲۱۷
تاریخ ارسال گزارش	۱۴۰۳۰۸۰۱۰

فهرست

- پاسخ ۱. سیستم پیشنهاد غذا..... ۱
- پاسخ ۲ - تبلیغات در یک فروشگاه آنلاین..... ۱۱
- پاسخ ۳ - موبایل اجتماعی..... ۱۶

شکل‌ها

- شکل ۱. تعداد انتخاب هر اکشن در الگوریتم Thompson sampling ۲
- شکل ۲. میانگین پاداش در الگوریتم Thompson Sampling ۲
- شکل ۳. مقدار Regret الگوریتم Thompson sampling ۳
- شکل ۴. نمودار توزیع بتای تخمین زده شده‌ی اکشن‌های مختلف در Thompson Sampling ۳
- شکل ۵. میزان Regret الگوریتم UCB به ازای c های مختلف ۴
- شکل ۶. تعداد انتخاب هر اکشن در الگوریتم UCB ۵
- شکل ۷. میانگین پاداش در الگوریتم UCB ۵
- شکل ۸. مقدار Regret الگوریتم UCB ۶
- شکل ۹. میزان Reward روش Epsilon greedy به ازای مقادیر مختلف و ثابت ϵ ۶
- شکل ۱۰. میزان Regret روش Epsilon greedy به ازای مقادیر مختلف و ثابت ϵ ۷
- شکل ۱۱. تعداد انتخاب هر اکشن در الگوریتم Epsilon-Greedy ۸
- شکل ۱۲. میانگین پاداش در الگوریتم Epsilon-Greedy ۸
- شکل ۱۳. مقدار Regret الگوریتم Epsilon-Greedy ۹
- شکل ۱۴. مقایسه میانگین پاداش‌ها در سه الگوریتم UCB، Thompson و Epsilon greedy ۹
- شکل ۱۵. تابع reward در نظر گرفته شده برای اکشن‌های ۳ الی ۶ ۱۲
- شکل ۱۶. مقایسه تعداد اکشن‌های انتخاب شده در سه الگوریتم ۱۳
- شکل ۱۷. مقایسه میانگین پاداش در سه الگوریتم ۱۴
- شکل ۱۸. مقایسه میزان Regret در سه الگوریتم ۱۵
- شکل ۱۹. شبه کد روش اول اپدیت کردن Policy در محیط Multi agent RL ۱۷
- شکل ۲۰. شبه کد روش دوم اپدیت کردن policy در محیط Multi agent RL ۱۸

پاسخ ۱. سیستم پیشنهاد غذا

صورت بندی مسئله (شماره دانشجویی ۸۱۰۱۰۲۲۱۷):

بر اساس الگوریتم گفته شده در صورت سوال،

تعداد اکشن (بازو): ۴

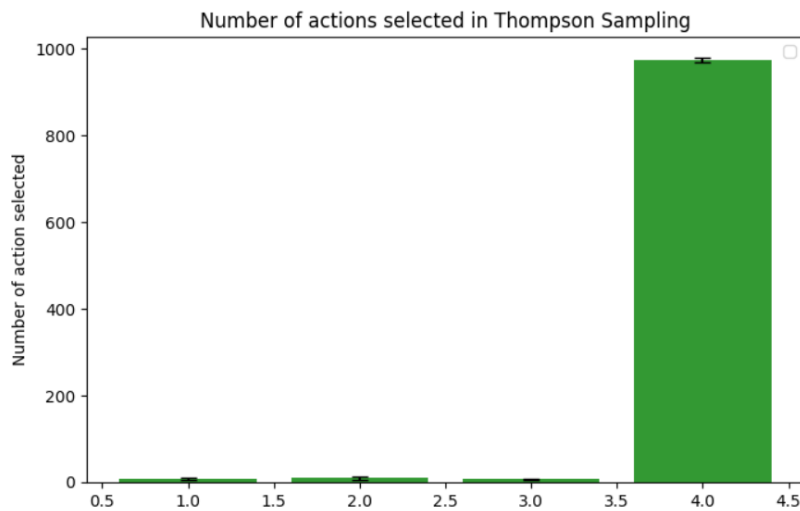
احتمال پاداش دادن هر اکشن (بازو): $[0.2, 0.2, 0.1, 0.7]$

درواقع انتخاب مناسب، اکشن ۴ است که بیشترین احتمال پاداش را دارد. ایجنت باید با استفاده از الگوریتم‌های مختلف، این اکشن بهینه را انتخاب کند.

در ادامه، الگوریتم‌های Thompson sampling، Upper Confidence Bar و Value-Based Epsilon Greedy هر کدام ۱۰ بار اجرا شده و نتایج مرتبط با هریک، یعنی میانگین پاداش، regret و همچنین تعداد دفعاتی که هر اکشن انتخاب شده است، ارائه خواهند شد. لازم به ذکر است که به دلیل رندوم بودن هر نمودار، confidence interval هریک نیز رسم شده است تا بتوان با اطمینان بالایی، بهترین الگوریتم این مسئله ی بخصوص n-arm bandit را یافت.

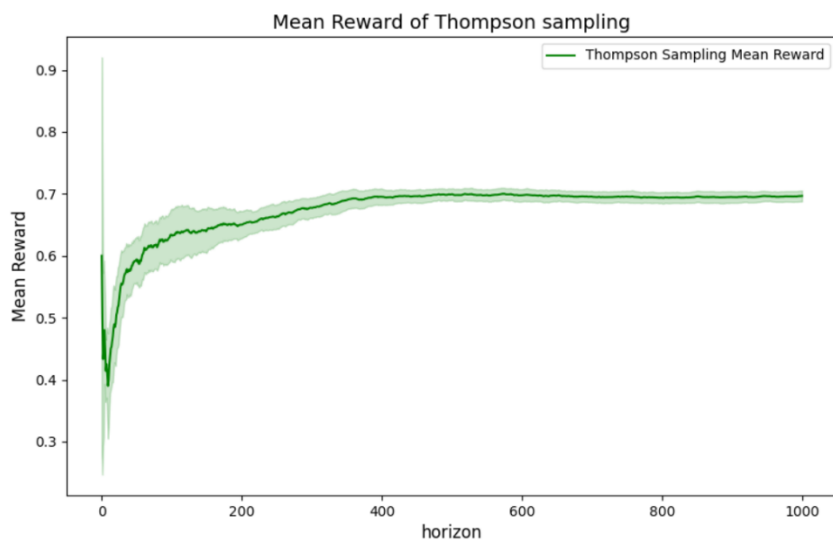
۱. Thompson Sampling:

برای پیاده‌سازی این الگوریتم از توزیع بتا (که شامل دو پارامتر α و β است) برای تخمین توزیع پاداش اکشن‌ها استفاده شده است. درواقع اگر یک اکشن، پاداش دهد ($\text{reward}=1$) آنگاه پارامتر α یک واحد افزوده شده و در غیر اینصورت به پارامتر β یک واحد افزوده می‌شود. نتایج پیاده سازی این الگوریتم در محیط شبیه سازی در ادامه قابل مشاهده است.



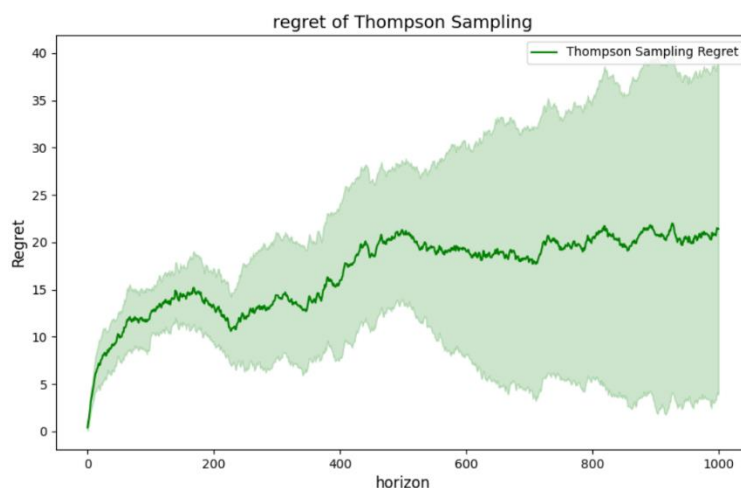
شکل ۱. تعداد انتخاب هر اکشن در الگوریتم **Thompson sampling**

همانطور که مشاهده می‌شود، تعداد انتخاب اکشن شماره ۴ (یعنی احتمال ۰.۷) بطور چشم گیری از سایر اکشن ها بیشتر است.



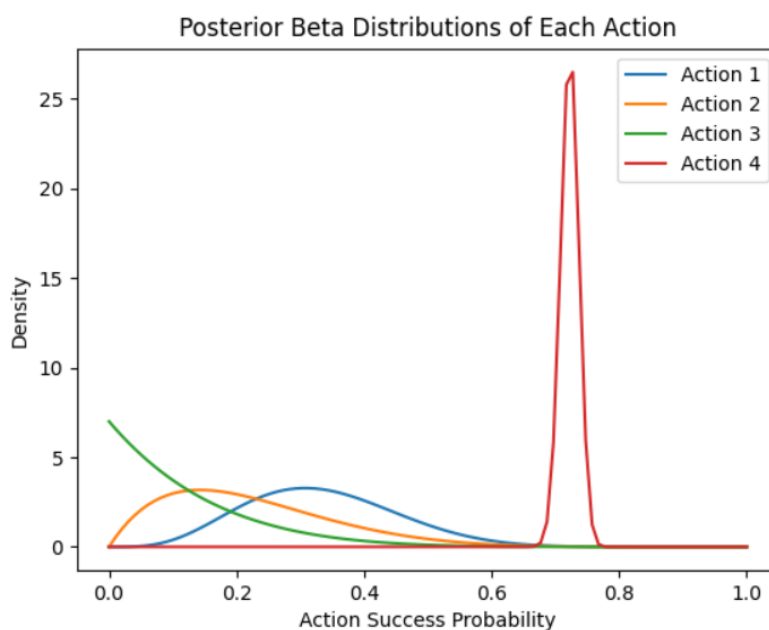
شکل ۲. میانگین پاداش در الگوریتم **Thompson Sampling**

میانگین پاداش ها بعد از حدود ۴۰۰ بار انتخاب اکشن ها، به احتمال پاداش بهترین اکشن همگرا شده است.



شکل ۳. مقدار **Regret** الگوریتم **Thompson sampling**

مشاهده می شود که میزان regret در حدود ۲۰ ثابت مانده است. یعنی ختلاف reward اکشن انتخاب شده، از بهترین اکشن بسیار کم بوده و نتوانسته است میزان regret را افزایش دهد. به بیان دیگر، پس از ۴۰۰ trial تقریباً در تمام trial ها، بهترین اکشن انتخاب شده است.



شکل ۴. نمودار توزیع بتای تخمین زده شده ی اکشن های مختلف در **Thompson Sampling**

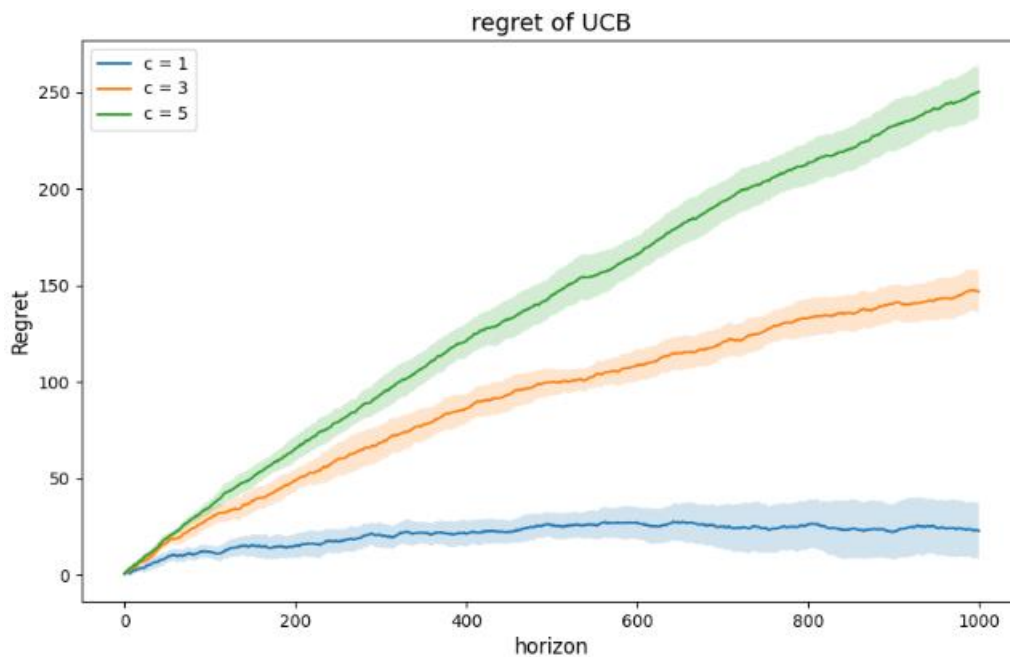
توزیع بتای اکشن ۴ به خوبی تخمین زده شده است اما سایر اکشن ها دقت خوبی ندارند. دلیل این اتفاق این است که الگوریتم به تعداد دفعات زیادی از اکشن ۴ نمونه برداری کرده است اما سایر انتخاب ها به دلیل کم بودن reward ها، به تعداد کمی انتخاب شده و بنابراین تخمین خوبی نیز از آن ها در دسترس نخواهد بود.

۲. Upper Confidence Bar:

الگوریتم UCB با استفاده از رابطه‌ی زیر، حد بالای مقادیر اکشن‌ها را محاسبه کرده و اکشنی را انتخاب می‌کند که بیشترین حد بالا را داشته باشد. حد بالا از رابطه زیر محاسبه می‌شود.

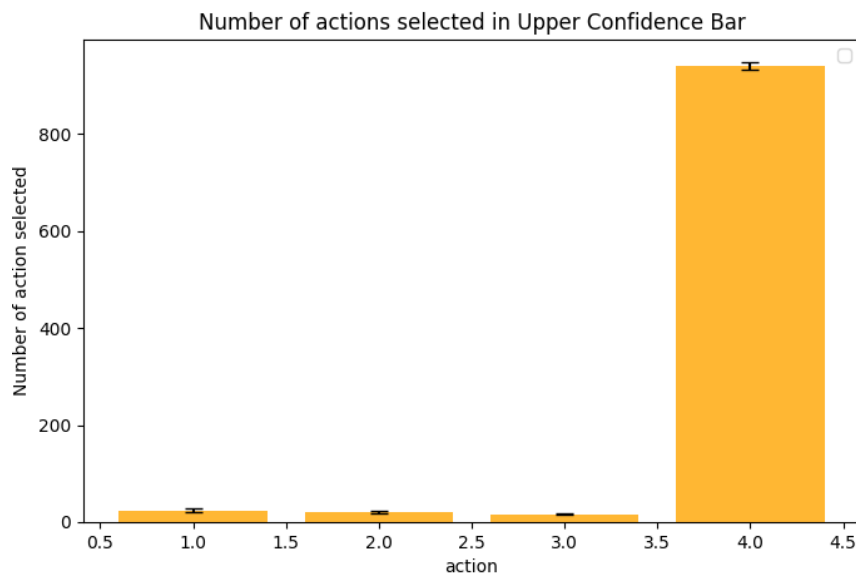
$$UCB_a = \hat{\mu}_a + c \cdot \sqrt{\frac{2 \ln t}{N(a)}}$$

این رابطه پیش از شروع الگوریتم، مقدار بینهایت را برای UCB هر اکشن اختصاص می‌دهد (چون $N(a)=0$). در واقع هر اکشن را یک بار انتخاب می‌کند و میزان μ_a هر اکشن را اپدیت می‌کند. در ادامه هر چه یک اکشن بیشتر انتخاب می‌شود، در دفعات بعدی احتمال کمتری برای انتخاب آن اکشن وجود دارد (چون $N(a)$ افزایش یافته است). درواقع یک تناسبی میان exploration و exploitation ایجاد می‌کند. الگوریتم پیاده‌سازی شده و نتایج در زیر قابل مشاهده است



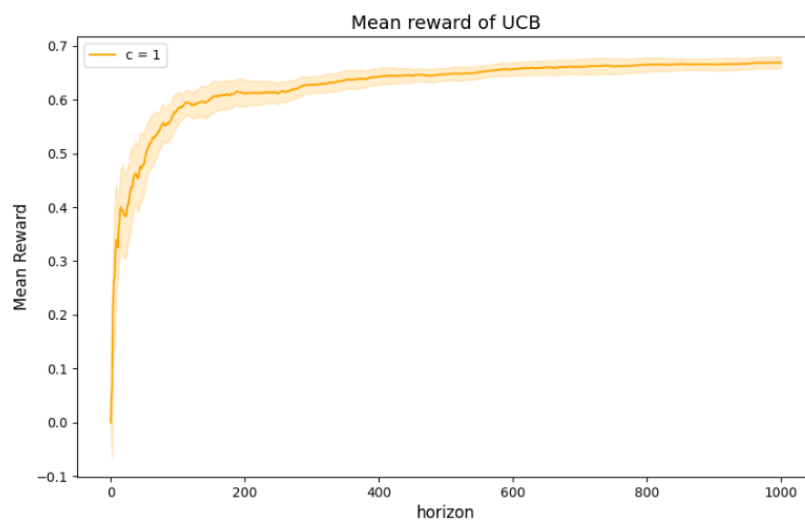
شکل ۵. میزان Regret الگوریتم UCB به ازای c های مختلف

با توجه به شکل فوق، بهترین انتخاب پارامتر، $c=1$ است. بنابراین در ادامه از همین مقدار برای رسم نمودارها استفاده می‌شود.



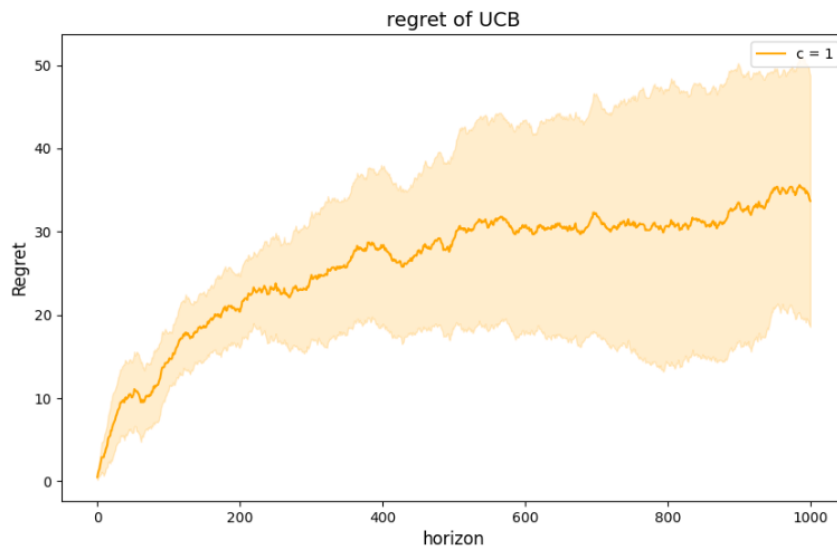
شکل ۶. تعداد انتخاب هر اکشن در الگوریتم UCB

مشاهده می شود که بهترین اکشن، به طور قابل توجهی از سایر اکشن ها بیشتر انتخاب شده است.



شکل ۷. میانگین پاداش در الگوریتم UCB

مشاهده می شود که میانگین پاداش به مقدار 0.7 (بیشترین پاداش ممکن) همگرا شده است.

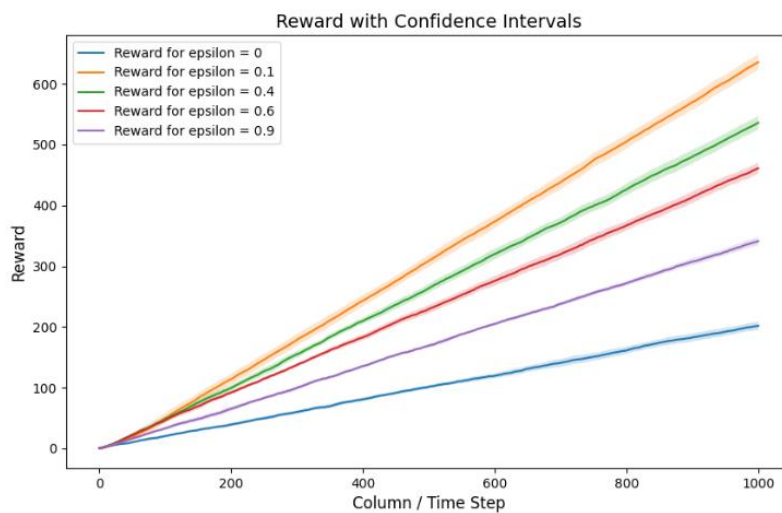


شکل ۸. مقدار **Regret** الگوریتم **UCB**

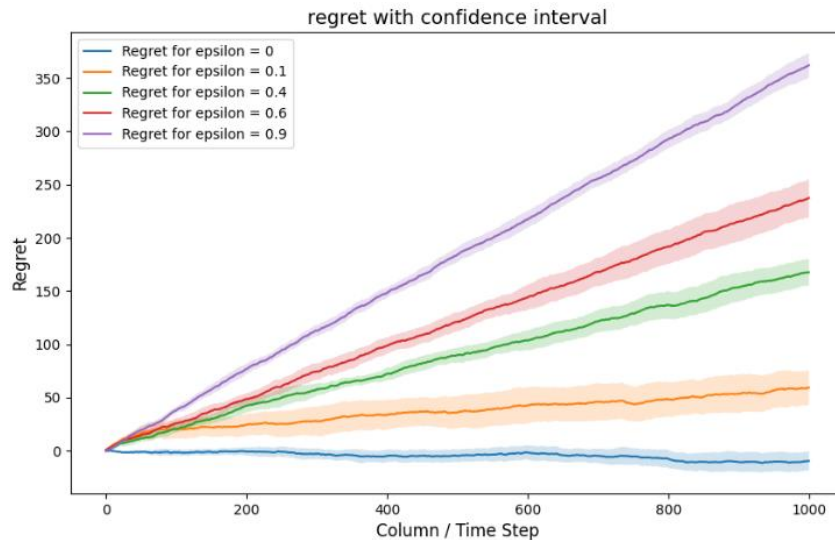
۱. Epsilon Greedy:

در این الگوریتم، در هر باری که اکشن انتخاب می‌شود، به احتمال ϵ ، این انتخاب رندوم خواهد بود.

اگر مقدار ϵ در طول یادگیری ثابت باشد، میزان **Reward** و **Regret** به ازای مقادیر مختلف ϵ به صورت شکل زیر خواهد بود.



شکل ۹. میزان **Reward** روش **Epsilon greedy** به ازای مقادیر مختلف و ثابت ϵ



شکل ۱۰. میزان **Regret** روش **Epsilon greedy** به ازای مقادیر مختلف و ثابت ϵ

از آنجایی که محیط در این مسئله ثابت است، اگر مقدار ϵ ثابت و کم باشد نتایج خوبی بدست خواهد آمد. درواقع با توجه به نتایج بدست آمده، بهترین مقدار ثابت برای ϵ ، 0.1 است.

اما بطور کلی و از آنجاییکه اغلب سیستم ها ایستا نیستند (مقادیر واقعی پاداش اکشن ها متغیر است) بهتر است پارامتر ϵ در طول یادگیری تغییر کند. درواقع در ابتدا باید میزان Exploration بسیار بیشتر از Exploitation باشد (یعنی انتخاب ها بطور غالب رندوم انتخاب شوند) اما با پیش رفتن یادگیری، به دلیل نزدیک شدن میانگین پاداش هر اکشن به میزان واقعی میانگین، باید Exploration کاهش یافته و الگوریتم به سمت Exploitation حرکت کند. به همین دلیل، مقدار ϵ در ابتدا برابر با 1 در نظر گرفته شده و با افزایش میزان یادگیری، کاهش یابد. البته مقدار ϵ نباید دقیقا برابر با 0 در نظر گرفته شود. چون ممکن است مقدار برخی از اکشن ها ایستا نبوده و تغییر کنند (البته در این مسئله این مورد وجود ندارد). بنابراین از تابع زیر برای مقدار دهی ϵ در هر trial استفاده می شود.

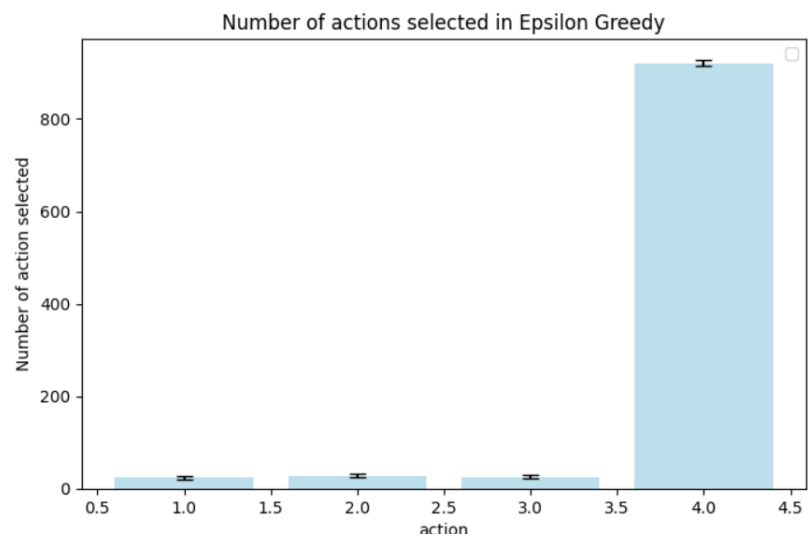
$$\epsilon = \min(\epsilon_{\min}, \epsilon_s \times 0.99^{\text{episode}})$$

که در آن،

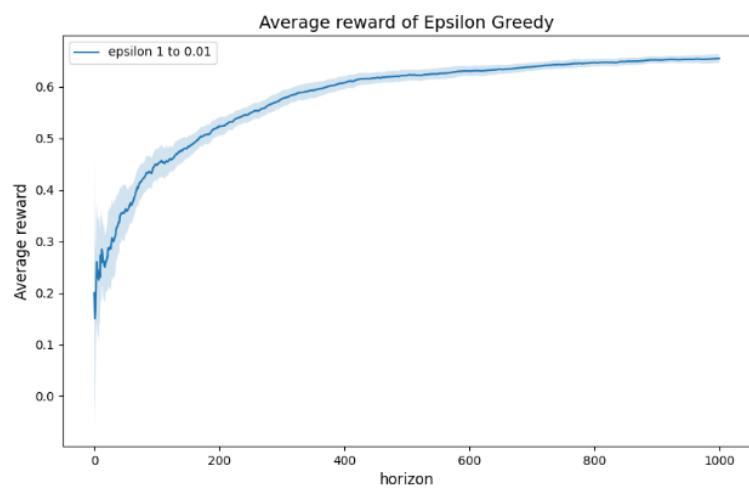
$$\epsilon_{\min} = 0.01$$

$$\epsilon_s = 1$$

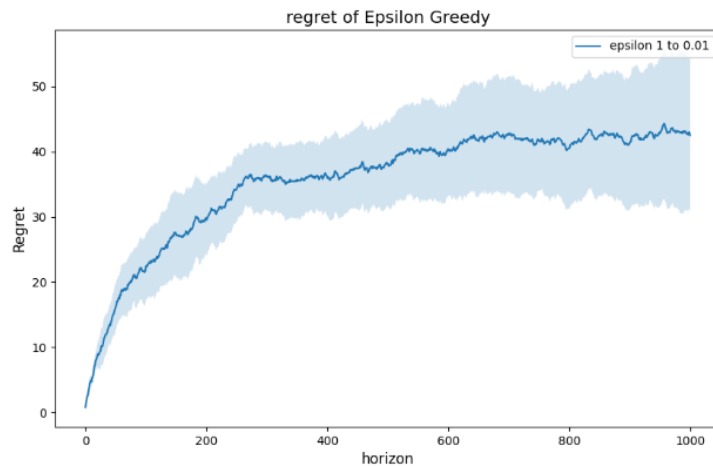
الگوریتم پیاده سازی شده و نتایج در ادامه ارائه خواهند شد.



شکل ۱۱. تعداد انتخاب هر اکشن در الگوریتم Epsilon-Greedy



شکل ۱۲. میانگین پاداش در الگوریتم Epsilon-Greedy

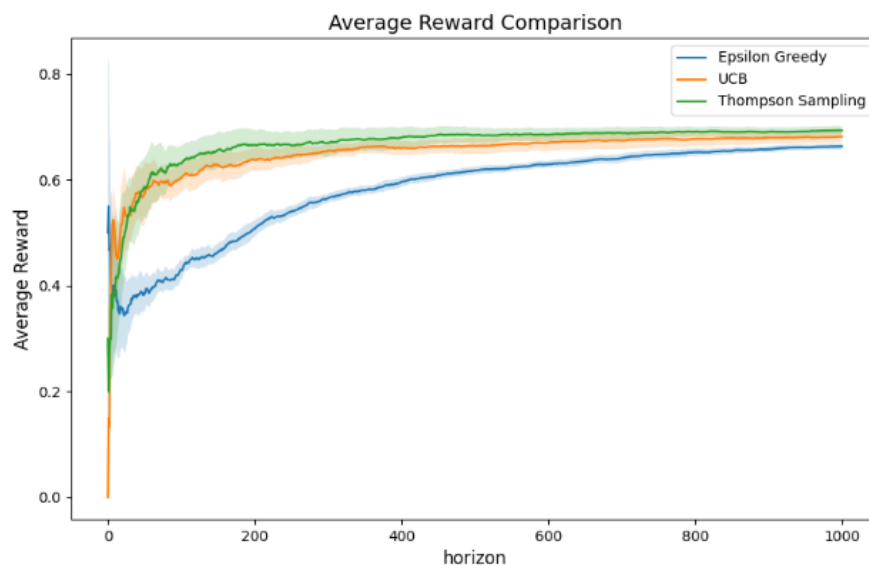


شکل ۱۳. مقدار **Regret** الگوریتم **Epsilon-Greedy**

مشاهده می شود که در این الگوریتم نیز، اکشن ۴ به طور چشم گیری بیشتر از سایر اکشن ها انتخاب شده است. همچنین میانگین پاداش نیز به 0.7 همگرا شده است.

میزان **Regret** در این الگوریتم صعودی خواهد. به این دلیل که با احتمال $\epsilon_{\min} = 0.01$ ، اکشن بصورت رندوم انتخاب می شود بنابراین در ۱ درصد مواقع، ممکن است اکشن غیر بهینه انتخاب شود. با این حال این الگوریتم نیز به خوبی عمل کرده است. درواقع در این مسئله که تابع پاداش ها ایستا است، بهتری است میزان $\epsilon_{\min} = 0$ انتخاب شود تا پس از تعدادی **trial**، تنها بهترین اکشن انتخاب شود.

در ادامه میزان میانگین پاداش این ۳ الگوریتم در یک نمودار رسم و مقایسه خواهند شد.



شکل ۱۴. مقایسه میانگین پاداش ها در سه الگوریتم **UCB**، **Thompson** و **Epsilon greedy**

مشاهده می شود که هر سه الگوریتم به خوبی عمل کرده اند. اما سرعت همگرایی الگوریتم Thompson Sampling بیشتر از سایرین بوده است. با توجه به نمودار فوق، ترتیب سرعت همگرایی الگوریتم‌ها در این مسئله‌ی خاص بصورت زیر خواهد بود.

Epsilon Greedy < UCB < Thompson Sampling

معیار دیگری مقایسه الگوریتم های فوق، محاسبه تعداد دفعاتی است که بهترین اکشن انتخاب شده است.

پاسخ ۲- تبلیغات در یک فروشگاه آنلاین

تعریف مسئله:

۶ اکشن به صورت زیر تعریف می‌شود:

$$a_1 = [ad1]$$

$$a_2 = [ad2]$$

$$a_3 = [ad1, ad2]$$

$$a_4 = [ad2, ad1]$$

$$a_5 = [ad1, ad1]$$

$$a_6 = [ad2, ad2]$$

درواقع اینکه کاربر کدام از تبلیغات را به چه ترتیب و چند بار انتخاب کند، هرکدام یک اکشن محسوب می‌شود.

در الگوریتم‌های n-arm bandit، همواره exploitation ادامه دارد و اگر که سلیقه‌ی مردم در دیدن تبلیغات و سپس خرید کردن به تدریج عوض شد، تبلیغات با ترتیب جدیدی برای مخاطب به نمایش در خواهد آمد. اما در روش AB testing این امکان وجود ندارد.

در مسئله‌ی بررسی شده، اگر روش AB testing اجرا شود، بطور مثال ۶ اکشن در ۶ صفحه‌ی مختلف قرار داده می‌شود و روی ۶۰۰ نفر تست می‌شود (هر اکشن ۱۰۰ نفر). سپس نرخ خرید در هر صفحه مشاهده می‌شود. سپس اکشنی که صفحه‌ی متناظر با آن، بیشترین نرخ خرید را داشته باشد، به عنوان اکشن بهینه معرفی شده و در ادامه نیز تبلیغات با همان ترتیب اکشن بهینه به نمایش کاربران درخواهد آمد. درواقع در AB testing در ابتدا Explore انجام می‌شود سپس تنها exploit انجام خواهد شد.

در مقابل، روش‌های n-arm bandit همواره مصالحه‌ای میان explore و exploit برقرار می‌کنند.

اکنون در این مسئله برای احتمال پاداش و تابع reward اکشن‌ها، فرض‌هایی در نظر گرفته می‌شود. برای a_1 و a_2 که تنها یک تبلیغات دیده شده است، هیچ reward داده نمی‌شود. درواقع به احتمال 100%، میزان پاداش برابر با 0 خواهد بود. اما برای سایر اکشن‌ها یعنی a_3 تا a_6 ، یک احتمالی در نظر گرفته می‌شود و با آن احتمال، پاداشی را از یک تابع نرمال با میانگین و واریانس مشخص تولید کرده و پاداش می‌دهد. احتمال پاداش دادن هر اکشن به صورت زیر در نظر گرفته شده است.

$$P(a_1) = 1$$

$$P(a_2) = 1$$

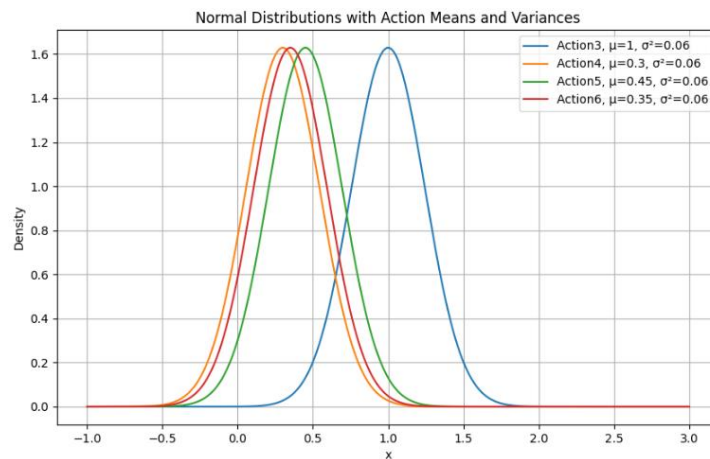
$$P(a_3) = 0.9$$

$$P(a_4) = 0.8$$

$$P(a_5) = 0.8$$

$$P(a_6) = 0.8$$

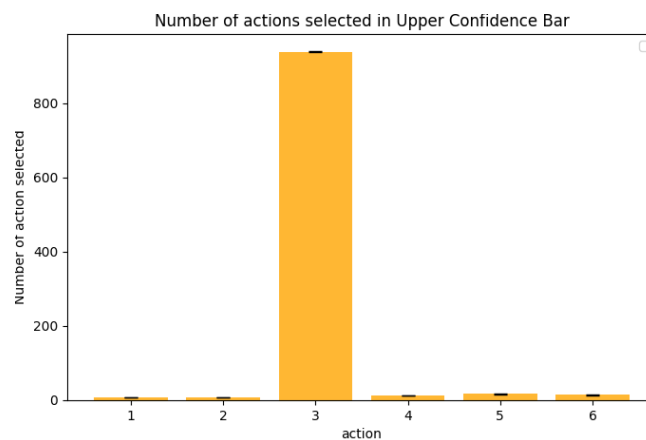
همچنین اگر قرار باشه برای هر یک از اکشن ها، پاداشی تولید شود، فرض می شود که میزان پاداش، از نمونه برداری از توابع نرمال زیر برای هر اکشن بدست می آید.

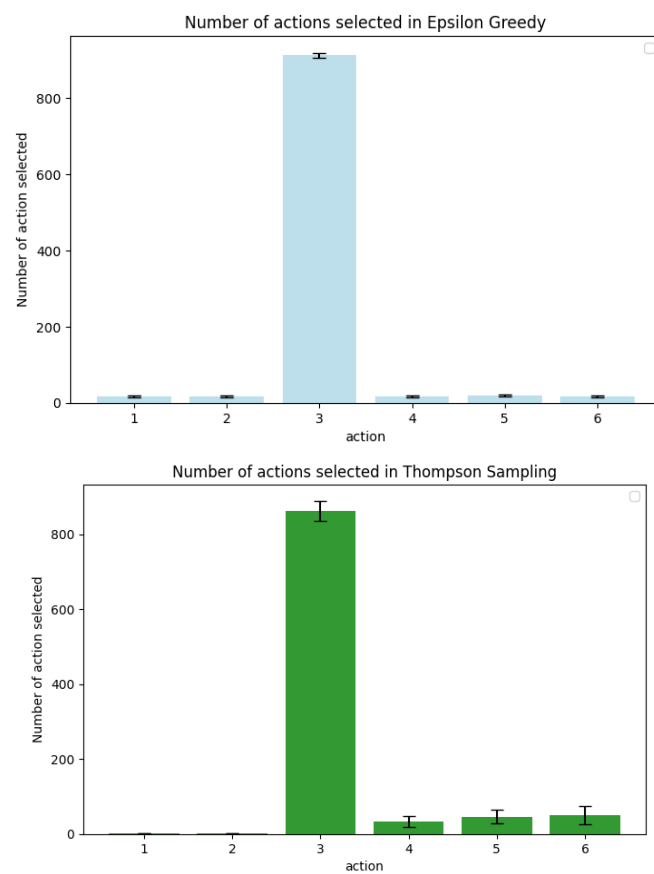


شکل ۱۵. تابع **reward** در نظر گرفته شده برای اکشن های ۳ الی ۶

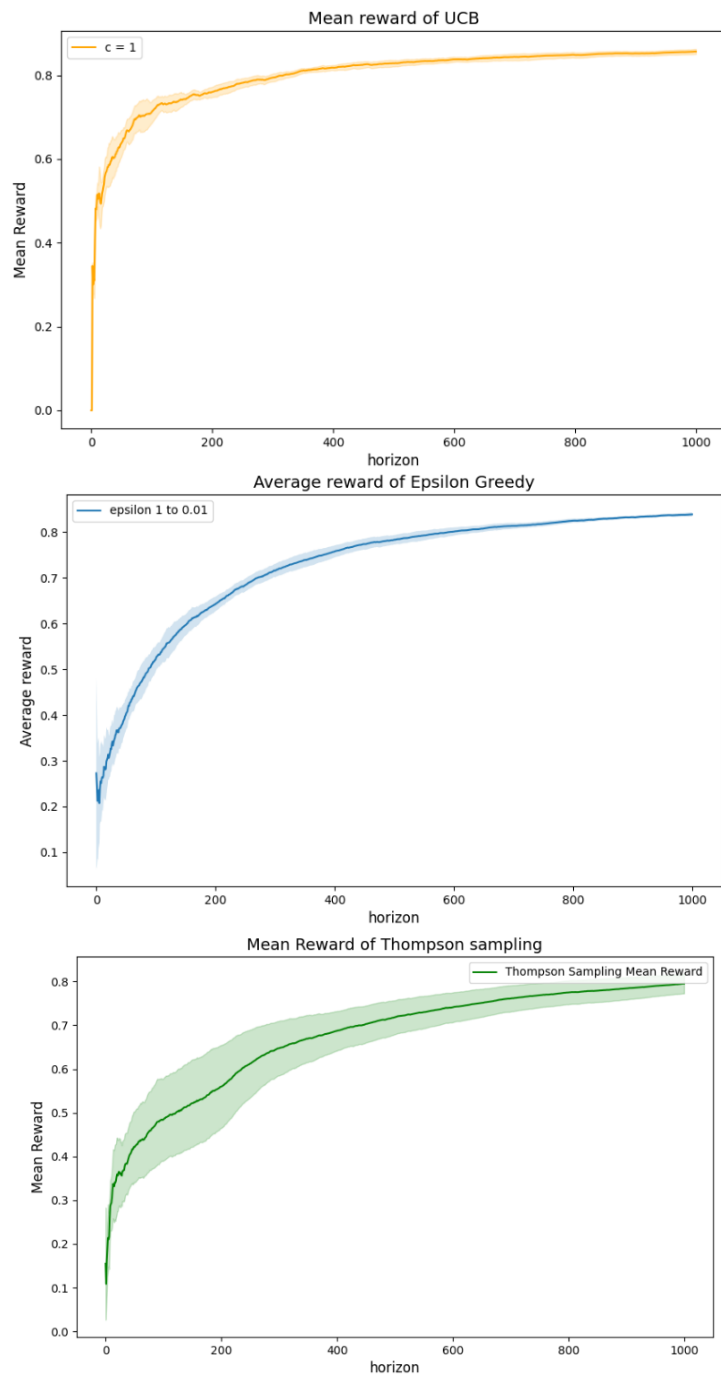
از شکل بالا مشخص است که برای a_3 بیشترین میزان میانگین در نظر گرفته شده است. بنابراین انتظار می رود در الگوریتم ها، a_3 بیشتر از سایر اکشن ها انتخاب شود.

در ادامه نمودارهای **reward**، **regret** و تعداد انتخاب اکشن ها توسط الگوریتم های **UCB** و **Epsilon greedy** و **Thompson sampling** ارائه خواهد شد.

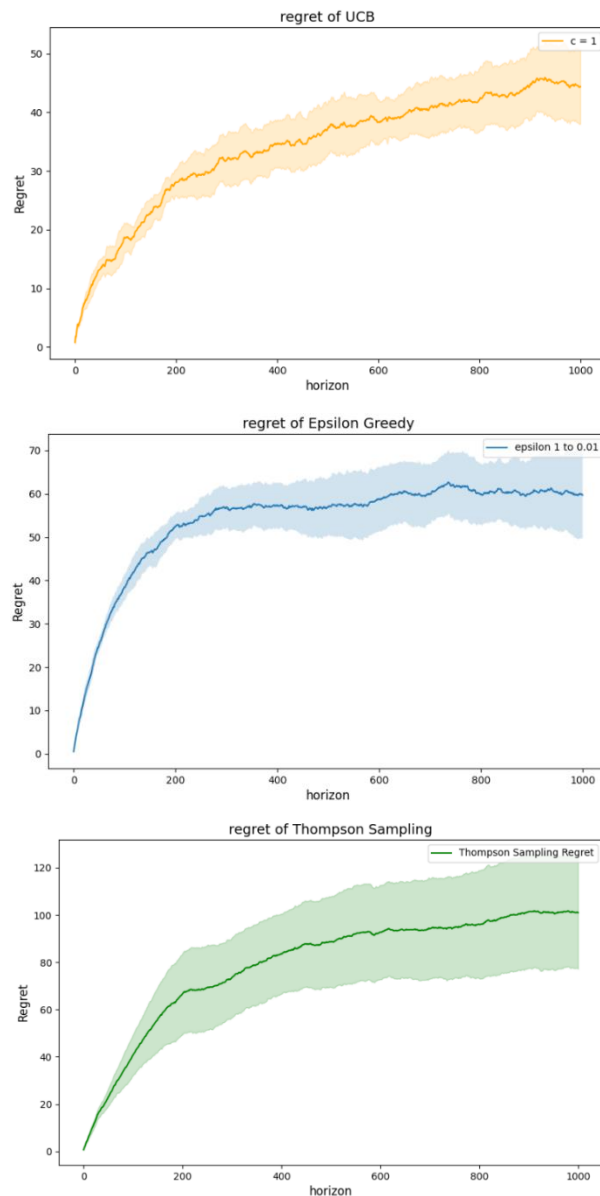




شکل ۱۶. مقایسه تعداد اکشن های انتخاب شده در سه الگوریتم.



شکل ۱۷. مقایسه میانگین پاداش در سه الگوریتم



شکل ۱۸. مقایسه میزان **Regret** در سه الگوریتم

مشاهده می شود که در هر سه الگوریتم، a_3 که برای آن احتمال پاداش و همچنین تابع reward با میانگین بالاتر در نظر گرفته شده بود، به تعداد بیشتری از سایر اکشن ها انتخاب شده است. در این مسئله ی خاصی که طراحی شد، الگوریتم UCB عملکرد بهتری داشته است. درواقع بطور میانگین سرعت همگرایی بالاتری و میزان regret کمتری را به خود اختصاص داده است. البته این اتفاق به طور میانگین رخ داده است و ممکن است به ازای برخی از دفعات اجرای الگوریتم ها، نتایج متفاوت شود.

پاسخ ۳ - موبایل اجتماعی

در این مسئله، موبایلی که در تعامل با من است، باید به بهترین نحو برای من کار بکند. من به طور تنک به موبایل پاداش می دهم. موبایل من در تعامل با دیگر موبایل ها نیز است بطوری که اکشن سایر موبایل ها را دیده اما اطلاعی از میزان پاداشی که دریافت کرده اند ندارد. در زیر الگوریتمی ارائه می شود که در آن، موبایل من تخمینی از policy موبایل های دیگر زده، طبق آن policy پیش رفته و Reward میگیرد. در نهایت با استفاده از بهترین policy سایر موبایل ها، مدل policy موبایل من نیز اپدیت می شود. الگوریتم:

تعداد نسبتاً زیادی agent وجود دارد. به دلیل اینکه این تعداد زیاد است، باید برخی از آن ها انتخاب شوند. همچنین پس از انتخاب برخی از آن ها باید قابل اعتماد بودن agent ها نیز بررسی شود (این عملیات در تابع sample_agents انجام گرفته است).

اکنون تخمینی از policy فعلی موبایل من، و با توجه به reward هایی که تا کنون گرفته است زده می شود. تا بتوان بعداً آن را تخمین زد.

سپس یک for روی agent های نمونه برداری شده زده می شود. در این for، ابتدا اکشن هایی که ان agent انجام داده است دریافت می شود. سپس بر اساس این اکشن ها، و با استفاده از یک function approximator مانند شبکه های عصبی (NN)، تخمینی از policy آن agent بدست می آید. سپس آن policy روی موبایل من پیاده می شود. تا زمانی که من در این policy جدید به تعداد کافی، به موبایل خود reward (چه مثبت چه منفی) نداده ام، این policy در موبایل من اجرا می شود. بعد از اینکه به تعداد کافی reward دریافت شد، ایین policy جدید ارزیابی می شود.

اگر میزان reward این policy جدید بیشتر میزان پاداش policy حال حاضر موبایل من بود، policy من اپدیت می شود (برای اپدیت کردن، می توان احتمال انتخاب هر اکشن را به نسبتی از احتمال های فعلی و احتمال های تخمین زده شده ی policy جدید اپدیت می کنیم)

اگر هم policy جدید به اندازه کافی از policy قبلی بهتر نبود، اطلاعات آن agent، در نظر گرفته نشده و به agent بعد رفته، اکشن ها را دریافت می کنیم و باقی مراحل را از سر میگیریم. شبهه کدی که الگوریتم گفته شده را پیاده می کند در زیر قابل مشاهده است.

```
# Main function to compare and learn policies from other agents
def improve_self_policy(self_agent, agents, num_sample_agents, min_required_rewards):
    # Sample a subset of agents from the pool
    sampled_agents = sample_agents(agents, num_sample_agents)
    # Evaluate the self agent's current policy
    self_current_rewards = update_self_reward(self_agent, self_agent.policy)
    self_reward_evaluation = evaluate_agent_policy(self_current_rewards)
    # Loop through each sampled agent to compare policies
    for agent in sampled_agents:
        # Get the actions taken by this agent
        agent_actions = get_agent_action(agent)
        # Estimate its policy using neural network (NN)
        agent_estimated_policy = estimate_agent_policy_by_NN(agent_actions)
        # Collect enough rewards using this estimated policy
        agent_rewards = []
        while not enough_rewards_received(agent_rewards, min_required_rewards):
            agent_rewards.append(use_agent_policy(agent_estimated_policy))
        # Evaluate the policy based on the rewards obtained
        agent_reward_evaluation = evaluate_agent_policy(agent_rewards)
        # If the sampled agent's policy is better, update self's policy
        if agent_reward_evaluation > self_reward_evaluation:
            # Update self's policy and reward evaluation
            update_self_policy(self_agent, agent_estimated_policy)
            self_current_rewards = update_self_reward(self_agent, self_agent.policy)
            self_reward_evaluation = evaluate_agent_policy(self_current_rewards)
    return self_agent # Return the updated self agent
```

شکل ۱۹. شبه کد روش اول اپدیت کردن Policy در محیط Multi agent RL

روش دیگر این است که در بازه های زمانی مختلف (این زمان به اندازه ای که مطمئن باشیم موبایل ها reward گرفته اند بزرگ است) اقدام به نمونه برداری کرده، نرخ استفاده از تک تک اکشن های n_i را اندازه گرفته و با توجه به این نرخ ها، policy را تغییر داد. درواقع احتمال انتخاب اکشن π را در موبایل خود، برابر قرار دهم با میانگین وزن داری از احتمال فعلی انتخاب اکشن π و نرخ انتخاب اکشن π در سایر agent ها.

هرچه reward هایی به اکشن های موبایل خود داده ام از اهمیت بیشتری داشته باشد، وزن policy فعلی (که فرض گرفته شده است در حال حاضر برای من نسبتا خوب عمل می کند.) بیشتر بوده و هرچه اهمیت اکشن و reward سایر agent ها بیشتر باشد، وزن نرخ اکشن های سایر agent ها بیشتر خواهد بود. مانند بخش قبل، شرطی قرار داده شده است که اگر پس از اپدیت، policy جدید عملکرد خوبی در مقابل من نداشت، به این معنی است که از اطلاعات اکشن سایر agent ها نمیتوان برای من استفاده کرد. درواقع برای

بهینه کردن policy در این حالت، نیاز به اطلاعات reward سایر agent ها نیز دارد تا بتوان بر اساس آن، وزن‌های متغیر با هر یک اکشن بصورت جداگانه اپدیت کرد.

این الگوریتم در تصویر زیر قابل مشاهده است.

```
def update_self_policyy(all_agents_action_rate, self_action_rate,
                        other_agents_importance_weight, self_agent_importance_weight):
    total_weight = other_agents_importance_weight + self_agent_importance_weight
    assert total_weight != 0, "The sum of importance weights must not be zero."
    aggregated_action_rates = (
        other_agents_importance_weight * sum(all_agents_action_rate) / len(all_agents_action_rate)
        + self_agent_importance_weight * sum(self_action_rate) / len(self_action_rate)
    )
    self_policy = aggregated_action_rates / total_weight
    return self_policy

def main_update_process(agents, self_agent, n, other_agents_importance_weight, self_agent_importance_weight):
    all_agents_action_rate = collect_all_agents_action_rates(agents, n)
    self_action_rate = get_agent_action_rates_for_n_steps(self_agent, n)

    previous_policy = self_agent.policy # Assuming self_agent has a current policy stored.
    previous_policy_score = evaluate_policy(previous_policy)

    new_policy = update_self_policyy(all_agents_action_rate, self_action_rate,
                                     other_agents_importance_weight,
                                     self_agent_importance_weight)

    new_policy_score = evaluate_policy(new_policy)

    if new_policy_score > previous_policy_score:
        self_agent.policy = new_policy # Update policy only if new policy is better.
    return self_agent.policy
```

شکل ۲۰. شبه کد روش دوم اپدیت کردن policy در محیط Multi agent RL